

A Conceptual Framework for Task Categorization and Dynamic Stress-Energy Modeling

1 Moodmeter / Lovemeter Extension

A future extension of the framework is the introduction of a mood-based prediction engine.

The system is intended to:

- Estimate a time-dependent mood value.
- Predict emotional development up to three weeks into the future.
- Combine mood and energy states to estimate task completion probability.
- Derive auxiliary behavioral statistics from the resulting state variables.
- Model emotional persistence, anticipation, and interaction-driven behavioral changes.

The temporal visualization of mood is intentionally nonlinear:

- Recent history is stretched approximately linearly to preserve short-term fluctuations.
- Distant past and future regions are compressed logarithmically.

This representation enables simultaneous inspection of immediate emotional fluctuations and long-term behavioral trends within a single visualization.

1.1 Emotional Point Definition

The central quantity of the Moodmeter subsystem is the *Emotional Point* value, denoted by

$$M$$

The Emotional Point represents the current emotionally weighted engagement state of the user.

Unlike the Energy Point system, which models physiological and cognitive capacity, the Emotional Point system models:

- emotional activation,
- motivational coloration,
- affective persistence,
- interpersonal anticipation,
- emotional reinforcement,
- and subjective narrative engagement with reality.

The nominal emotional baseline is defined as

$$M_{\text{base}} = 1000$$

corresponding to a psychologically neutral but emotionally healthy state.

1.2 Emotional State Ranges

The following qualitative state regions are defined:

Table 1: Qualitative emotional state ranges.

Range	Interpretation
$M < 150$	Colorless / emotionally depleted state
$150 \leq M < 700$	Moody / emotionally suppressed state
$700 \leq M < 1500$	Normal emotionally active state
$1500 \leq M < 10000$	Elevated emotional activation / crush state
$10000 \leq M < 10^6$	Severe emotional amplification state
$M \geq 10^6$	Emotionally supercritical state

2 Mood Prediction and Temporal Visualization Framework

2.1 Conceptual Overview

The Moodmeter/Lovemeter subsystem extends the existing Energy Point framework by introducing a continuous emotional-state estimation and prediction engine.

The objective of the subsystem is not merely to display historical emotional events but to construct a smooth, interpretable emotional trajectory capable of short-term forecasting.

The framework separates the visualization problem into three distinct layers:

1. Historical event storage

2. Historical interpolation and smoothing
3. Future extrapolation and prediction

The resulting graph therefore combines:

- Exact reconstruction of historical emotional events
- Smooth interpolation between recorded events
- Dynamically evolving future prediction curves

2.2 Historical Data Representation

All emotional events are stored as the so called Data points (to be distinguished from the helper points used to construct a continuous graph).

Each event is represented as

$$(t_i, y_i)$$

where:

- t_i denotes the real timestamp
- y_i denotes the emotional state value

Data points defines the interpolation geometry. Only data points are saved. After interpolation, a helper array consisting of helper points in 1-minute jumps is calculated for the determined time range of -5 to +21 days. To save on computing and rendering power, the helper array is saved once and reused for future loads. It is only recalculated once per day or when a change to the Data points takes place

The temporal axis distortion described later in this document is applied exclusively during rendering and visualization.

The original data therefore remain physically and temporally consistent regardless of graphical transformation.

2.3 Event-Based Emotional Perturbations

Emotional states evolve through discrete event perturbations. A new data point is defined as

$$(t_i, M_i)$$

for which:

- t_i denotes the timestamp,

- M_i denotes the newly calculated emotion points, calculated using the following formula

$$M_i = \overline{M}(t_i) + \Delta M$$

$\overline{M}(t_i)$ is the value of emotion point at the time t_i before the change takes place. It is retrieved from the helper array.

M_i is the new emotion points. The point (t_i, M_i) is stored as a data point. The helper point is automatically updated during the recalculation.

2.4 Single Event Definitions

A preset of single events can be used to register a new data point with the modified emotion point at the moment of applying. An event is defined by the event name and its effect on the score ΔM . New events can be manually created and is added to the preset upon creation.

2.5 Eye-Contact Interaction Model

Eye-contact interactions are modeled as duration-dependent high-impact perturbations.

The resulting emotional gain is defined as

$$\Delta M_{\text{eye}} = r_{\text{eye}} \cdot t_{\text{eye}}$$

where:

- r_{eye} denotes eye-contact reactivity,
- t_{eye} denotes eye-contact duration.

The eye-contact reactivity coefficient itself depends on the current emotional state:

$$r_{\text{eye}} = \lambda(M - M_0)$$

where:

$$M_0 = 700$$

and the default for λ is $\lambda = 0,0001$.

The formulation therefore models the observation that emotionally amplified states become increasingly sensitive to emotionally significant interaction events.

2.6 Manual Emotional Overrides

The subsystem additionally supports manual emotional correction events. These events intentionally override automatically estimated states in order to account for subjective interpretation effects not directly inferable from external interaction data. Overriding at the time t_i immediate creates the data point (t_i, M_i) in which

$$M_i = M_{\text{override}}$$

An override can be applied to any time point, not necessarily the moment of override input.

2.7 Representation Of Data

The calculated data is represented in a M(t)-Diagram. The rendering concept is similar to that of Stressmeter.

2.8 Temporal Axis Distortion

A central design requirement of the visualization system is the simultaneous display of:

- Fine short-term emotional fluctuations
- Long-term future predictions extending over multiple weeks

A purely linear temporal axis is unsuitable for this purpose because near-term details would become visually compressed.

To address this limitation, a nonlinear temporal distortion function is introduced.

The graph coordinate x is related to real elapsed time t through

$$t = \alpha x^5$$

with default parameter

$$\alpha = 0.0003$$

Solving for the graph coordinate yields

$$x = \left(\frac{t}{\alpha} \right)^{1/5}$$

This transformation produces the following behavior:

- Extremely high temporal resolution near the present moment
- Progressive compression of distant future regions
- Continuous and smooth transition between both regimes

The transformation therefore behaves similarly to a “time microscope” near the present while simultaneously functioning as a “time telescope” for long-range prediction.

To ensure numerical stability near

$$t = 0$$

a small clamping constant

$$\varepsilon$$

is introduced:

$$x = \text{sign}(t) \left(\frac{|t| + \varepsilon}{\alpha} \right)^{1/5}$$

The parameter ε is typically chosen as one minute.

2.9 Axis Tick Generation

Tick marks define the marking on the x-axis of the graph. The standard markings are -4 days, -2 days, 00:00(-1), 12:00(-1), 00:00, 06:00, 12:00, 18:00, +1 day, +2 days, +4 days, +10 days, +20 days.

2.10 Historical Interpolation

The historical graph is reconstructed using spline interpolation.

The selected interpolation method is the centripetal Catmull–Rom spline.

The method was selected due to the following properties:

- Exact interpolation through all recorded points
- Smooth first-order continuity
- Local behavior
- Numerical stability
- Minimal oscillatory artifacts

Unlike high-degree global polynomial interpolation, spline interpolation avoids large-scale oscillations and ensures that local point adjustments affect only nearby graph regions.

The interpolation therefore reconstructs a smooth emotional trajectory while preserving exact historical event values.

Interpolation is performed in real timestamp space rather than transformed graph space.

The rendering pipeline is therefore:

$(t_i, y_i) \rightarrow$ spline interpolation $\rightarrow$ temporal distortion transform $\rightarrow$ screen coordinates

2.11 Current State Estimation

The prediction engine operates using two continuously estimated state variables:

$x(t)$: Current emotional state
 $m(t)$: Current emotional slope/trend

The emotional state is extracted directly from the interpolated graph.

The slope is estimated locally using recent graph points.

The system therefore interprets repeated positive or negative emotional developments as persistent trends rather than isolated fluctuations.

2.12 Prediction Model

The future prediction engine assumes that emotional trajectories evolve continuously rather than changing instantaneously.

The extrapolation model is based on the following principles:

- Existing emotional momentum initially persists
- Emotional trajectories gradually return toward a baseline state
- The strength of baseline attraction depends on emotional intensity
- High emotional states exhibit increased persistence
- The return toward baseline becomes progressively weaker near equilibrium

The model therefore introduces:

$$B$$

as the emotional baseline.

The predicted state evolves according to

$$\frac{dx}{dt} = m$$

where m denotes the current emotional slope.

The target slope directing the system back toward baseline is defined as

$$m_{\text{target}} = -k(x - B)$$

where:

- k is the baseline return strength
- $x - B$ represents distance from emotional equilibrium

Unlike a purely linear relaxation system, emotional persistence increases with emotional magnitude.

The persistence factor is therefore modeled as

$$P(x) = 1 + \left(\frac{|x|}{1000} \right)^{1.2}$$

The effective stabilization coefficient becomes

$$k_{\text{effective}} = \frac{k_{\text{base}}}{P(x)}$$

This formulation resolves the instability conflict of the purely linear stabilization model by ensuring that large emotional states decay progressively more slowly rather than collapsing unrealistically toward baseline.

The current slope gradually approaches the target slope according to

$$\frac{dm}{dt} = p(x) (m_{\text{target}} - m)$$

where

$$p(x) = \frac{p_0}{P(x)}$$

is the persistence-adjusted directional adaptation coefficient.
Higher emotional states therefore produce:

- stronger emotional inertia,
- slower directional reversal,
- prolonged afterglow behavior,
- and persistent trend continuation.

The resulting behavior yields:

- Continued short-term movement along the current trajectory
- Smooth peak formation
- Gradual decline toward baseline
- Asymptotic stabilization near equilibrium

The model therefore avoids unrealistic instantaneous emotional reversals.

2.13 Derived Behavioral Attributes

The emotional state generates secondary behavioral attributes used throughout the simulation framework.

Representative derived attributes include:

- motivation,
- social confidence,
- focus stability,
- music amplification,
- notification sensitivity,
- replay-loop probability,
- coincidence significance bias,
- outfit optimization,
- cinematic perception amplification,
- and train-window delusion.

Representative formulations include:

$$A_{\text{motivation}} = \alpha(M - 700)$$

$$A_{\text{social}} = \beta(M - 700)$$

$$A_{\text{music}} = 1 + \frac{M - 700}{700}$$

$$A_{\text{train}} = \gamma(M - 700)$$

Although emotional activation strongly increases motivation, high emotional states may simultaneously reduce sustained deep-focus capability.

Focus stability is therefore modeled nonlinearly.

This formulation allows emotionally amplified states to increase:

- social behavior,
- anticipation,
- environmental romanticization,
- and motivational activation,

while simultaneously reducing:

- deep-focus stability,
- abstract cognitive endurance,
- and uninterrupted study capability.

2.14 Task Completion Coupling

Task completion probability is determined jointly from:

- current energy state,
- emotional state,
- distraction level,
- and task structure.

Structured externally constrained tasks remain relatively stable even during high emotional activation, whereas internally self-regulated deep-focus tasks become increasingly unstable.

The resulting task-completion framework therefore permits:

- high social functionality during emotional amplification,
- preserved attendance behavior,
- increased motivational activation,
- but degraded deep-focus cognitive performance.

2.15 Prediction Characteristics

The resulting prediction curve exhibits the following qualitative behavior:

1. Positive slopes initially continue upward
2. The upward trend gradually weakens
3. A smooth local maximum is formed
4. The trajectory subsequently declines
5. The decline progressively flattens near baseline

This behavior produces visually smooth and psychologically plausible emotional trajectories.

2.16 Simulation Resolution

Prediction curves are evaluated numerically using fixed discrete timesteps.

The default temporal resolution is one minute.

Despite the high simulation resolution, only rendered screen-space samples are visualized.

The implementation therefore distinguishes between:

- Stored event points
- Internal simulation states
- Rendered visualization samples

This separation minimizes storage overhead while preserving visual smoothness.

2.17 Rendering Architecture

The rendering pipeline is summarized as follows:

1. Load historical emotional events
2. Construct spline interpolation
3. Estimate current emotional state and slope
4. Construct helper array
5. Simulate future prediction trajectory
6. Apply temporal axis distortion
7. Generate screen coordinates
8. Render graph and diagnostic overlays

The final visualization therefore combines:

- Exact historical reconstruction
- Smooth continuous interpolation
- Dynamic future prediction
- Nonlinear temporal visualization

within a unified graph representation.

3 Software Design Requirements

The implementation should satisfy the following software-engineering principles:

- Clear separation of concerns
- Modular architecture
- Readable and maintainable code
- Minimal external dependencies
- Extensible parameter tuning
- Robust time parsing utilities
- Pure functions where feasible

All adjustable parameters should be centralized for efficient calibration and experimentation.